

20250612日报

今日工作内容

1、学习version_proxy包源码，并通过阅读fc_mall_front项目了解请求流程；

1. fc_mall_front项目中的新品页流程：main.go->main_process.go(根据请求中的bid选择到新品页请求)->新品页下的main.go的Handle()(进入主逻辑处理入口)->获取候选者（包括英雄（GetHero()）、皮肤（GetSkin()）、表情（GetEmoji()）等）->无兜底情况时会调用handle()(推荐处理，比如删除预热皮肤（还没卖的）、用模型进一步推荐，从而提升推荐效果)->推荐结果以日志形式输出。
2. 简单调用version_proxy包，输出一些指定的英雄属性

```
1  var sceneVal version_proxy.HeroScene = 1
2      name := sceneVal.String()
3      fmt.Println("HeroScene: ", name)
4      var val2 version_proxy.HeroCommercialTag = 1
5      status := val2.Descriptor()
6      fmt.Println(status)
7      heroreq := version_proxy.HeroRequest{
8          ClientVersion: 1,
9          Scene:         sceneVal,
10         ClientSvn:     1,
11     }
12     version := heroreq.GetClientVersion()
13     svn := heroreq.GetClientSvn()
14     fmt.Printf("ClientVersion=%d, ClientSvn=%d\n", version, svn)
```

```
HeroScene: HeroScene_Recent
EnumDescriptor{Syntax: proto3, FullName: version_proxy.HeroCommercialTag, Values: [{Name: HeroCommercialTag_None}, {Name: HeroCommercialTag_WarmUp, Number: 1}, {Name: HeroCommercialTag_CanBuy, Number: 2}, {Name: HeroCommercialTag_Encore, Number: 3}, {Name: HeroCommercialTag_Discount, Number: 4}]}
ClientVersion=1, ClientSvn=1
```

2、学习kafka原理，并trpc-go 接入kafka插件（这里接入的是本地kafka数据源）；

kafka原理：kafka实例（一个实例通常就是一个broker，多个broker组成集群）作为中间的消息队列，让上下游服务完全解耦，并且对于上游消费者可以根据不同的消费者组进行负载均衡。对于生产者发送的消息是有类别的，每一个消息类对应一个topic，而每一个topic对应多个分区，每个分区可以在不同的实例上，从而实现分布式。每一个分区可以有多个副本，也就是follower，为了高可用性。

1. 消费者代码和对应的yaml文件：

```
1 func main() {
2     // 把消费者服务注册到服务器上，trpc-go框架会自动管理消费者协程等
3     s := trpc.NewServer()
4     kafka.RegisterKafkaConsumerService(s.Service("trpc.kafka.consumer.service"),
5     &consumer{})
6     if err := s.Serve(); err != nil {
7         log.Errorf("serving :%v", err)
8     }
9 }
10 type consumer struct{}
11
12 func (consumer) Handle(ctx context.Context, msg *sarama.ConsumerMessage)
13 error {
14     if rawContext, ok := kafka.GetRawSaramaContext(ctx); ok {
15         log.Infof("InitialOffset: %d", rawContext.Claim.InitialOffset())
16     }
17     log.Infof("get kafka message topic: %s, partition: %d, key: %s, value: %s",
18     string(msg.Topic), msg.Partition, string(msg.Key), string(msg.Value))
19     return nil
20 }
```

```
1  global:                #全局配置
2    namespace: Development    #环境类型，分正式Production和非正式
    Development两种类型
3    env_name: test          #环境名称，非正式环境下多环境的名称
4
5  server:
6    service:
7      - name: trpc.kafka.consumer.service
8        address: 127.0.0.1:9092?topics=kafka_test&group=kafka_test-group
9        protocol: kafka
10
11  plugins:                #插件配置
12  log:                    #日志配置
13    default:              #默认日志的配置，可支持多输出
14      - writer: console    #控制台标准输出 默认
15      level: debug        #标准输出日志的级别
```

2. 生产者的代码和yaml文件

```
1 func main() {
2     cfg, err := trpc.LoadConfig("trpc_go.yaml")
3     if err != nil {
4         panic("load config fail: " + err.Error())
5     }
6     if err := trpc.SetupClients(&cfg.Client); err != nil {
7         panic("failed to setup client: " + err.Error())
8     }
9     proxy := kafka.NewClientProxy("trpc.kafka.producer.service")
10    if err := proxy.Produce(context.TODO(), []byte("name"), []byte("drewjli")); err
    != nil {
11        panic(err)
12    }
13    // 生产原生 sarama 消息, 返回 offset、partition
14    partition, offset, err := proxy.SendSaramaMessage(context.TODO(),
sarama.ProducerMessage{
15        Topic:    "kafka_test",
16        Partition: 1,
17        Key:      sarama.ByteEncoder("age"),
18        Value:    sarama.ByteEncoder("24"),
19    })
20    if err != nil {
21        panic(err)
22    }
23    log.Info("partition, offset = ", partition, offset)
24
25    // 批量生产原生 sarama 消息
26    err = proxy.SendSaramaMessages(context.Background(),
[]*sarama.ProducerMessage{
27        {
28            Topic:    "kafka_test",
29            Partition: 2,
30            Key:      sarama.ByteEncoder("home"),
```

```

1  global:                                #全局配置
2      namespace: Development            #环境类型，分正式Production和非正式
Development两种类型
3      env_name: test                    #环境名称，非正式环境下多环境的名称
4
5  client:
6      service:
7          - name: trpc.kafka.producer.service
8            target: kafka://127.0.0.1:9092?topic=kafka_test
9
10     plugins:                            #插件配置
11     log:                                #日志配置
12         default:                        #默认日志的配置，可支持多输出
13             - writer: console           #控制台标准输出 默认
14             level: debug

```

开启kafka实例后，分别运行消费者和生产者程序，得到结果：

```

[root@drewjili-1t708sepa consumer]# go run
plugin log-default: setup success, time elapsed: 93.708us
2025-06-12 16:27:30.011 DEBUG maxprocs/maxprocs.go:48 maxprocs: Updating GOMAXPROCS=16: determined from CPU quota
2025-06-12 16:27:30.012 INFO server/service.go:207 process: 1093819, kafka service: trpc.kafka.consumer.service launch su
cess, tcp: 127.0.0.1:9092?topic=kafka_test&group=kafka_test group, serving ...
2025-06-12 16:27:30.124 INFO consumer/main.go:26 InitialOffset: 16
2025-06-12 16:27:30.124 INFO consumer/main.go:28 get kafka message topic: kafka_test, partition: 0, key: name, value: d
rewjili
2025-06-12 16:27:30.124 INFO consumer/main.go:26 InitialOffset: 16
2025-06-12 16:27:30.124 INFO consumer/main.go:28 get kafka message topic: kafka_test, partition: 0, key: age, value: 24
2025-06-12 16:27:30.124 INFO consumer/main.go:26 InitialOffset: 16
2025-06-12 16:27:30.124 INFO consumer/main.go:28 get kafka message topic: kafka_test, partition: 0, key: home, value: C
DC
2025-06-12 16:27:30.124 INFO consumer/main.go:26 InitialOffset: 16
2025-06-12 16:27:30.124 INFO consumer/main.go:28 get kafka message topic: kafka_test, partition: 0, key: school, value:
UR51C

[root@drewjili-1t708sepa producer]# go run
2025-06-12 16:27:29.354 INFO producer/main.go:34 partition, offset = 0 17
[root@drewjili-1t708sepa producer]#

```